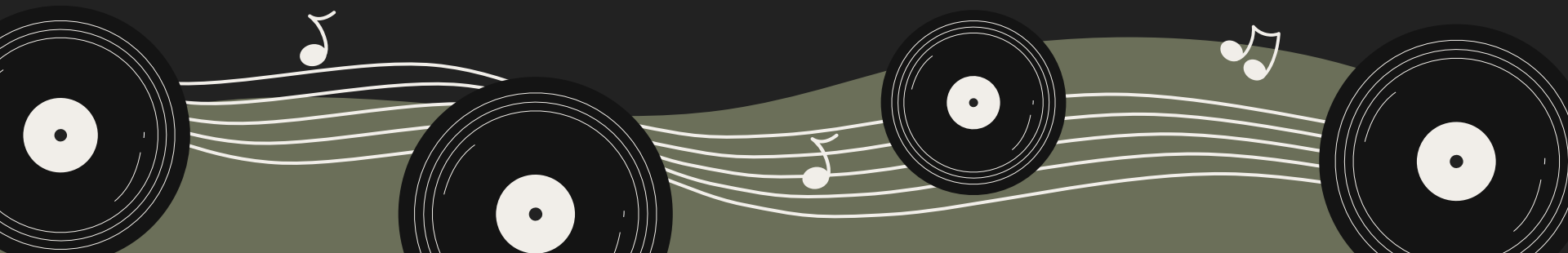


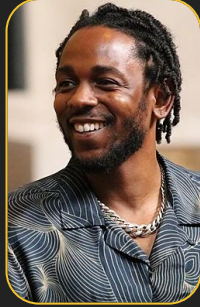


# Lyrical Layor

Kendrick Lamar and Taylor Swift Lyric Predictor & Game

Shrey and Rohan





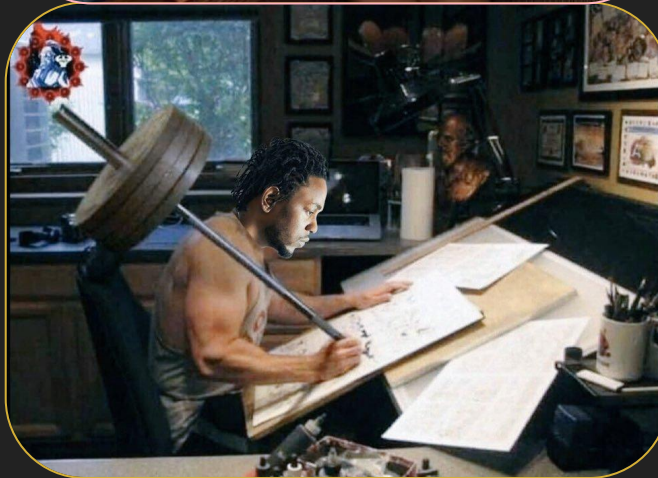
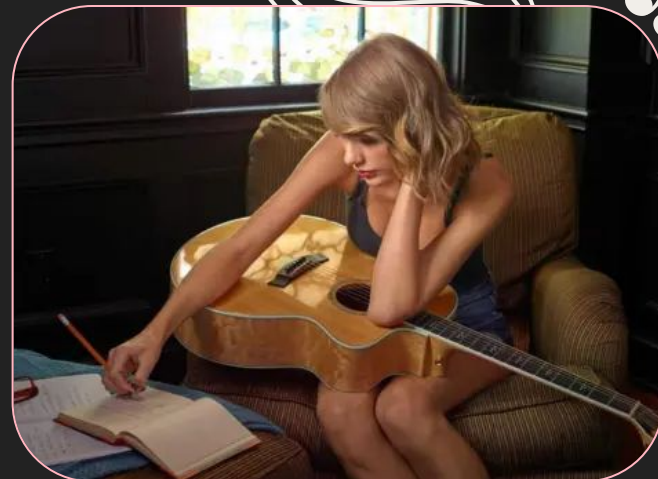
# Description

The Lyrical Laylor is a predictive writing assistant that allows users to compose lyrics in a similar style to that of either Taylor Swift and Kendrick Lamar. Using authentic song lyrics taken from custom made databases, this writing tool suggests the most likely next character and word based on the selected artist's pattern. This allows users to generate lyrics that are known to work in the space. As well as this, it includes an interactive game for when one gets bored composing. In said game the user must guess missing words taken from actual lyrics, which provides a fun way to decompress after a days worth of writing lyrics. The Lyrical Laylor is powered by a Trie dataset which ensures the reliability of this tool, making it the top choice for artists who are trying to break into the space!



# Features

- Toggle button between Kendrick Lamar and Taylor Swift style
- Displays next most likely character and word
- Displays top 5 next word predictions with percentages
- Real-time highlighting to determine if whatever the user is typing exists in our database (green=yes, red=no)
- Interactive lyric guessing game for fun
- Clean tailored graphical interface depending on the artist selected
- Designed to give an output as efficient as possible with a Trie





# Data Collection

- In order for our project to run, we had to make two large datasets for each artist containing all of their lyrics
- To collect this data we had search online for lyric compilations for each artist and copy and paste each into a .txt file
- Using an online find and replace tool, we had replaced all special characters, punctuation, and anything not directly related to the lyrics the artist wrote
- To keep it family friendly, we had also replaced all curse words with nicer alternatives to promote kindness



# Loading Data 🎵

01

.TXT

## Read File

```
Files  
.readAllLines(  
  Path.of(  
    "taylor.txt"))c
```

Reads every raw lyric line  
from the .txt file using  
NIO



02

SKIP

## Skip Headers

```
line.isEmpty()  
||  
line.startsWith("[")
```

Blank lines and section  
tags like [Verse 1] are  
filtered out



03

CLNR

## Clean Text

```
replaceAll(  
  "[^a-zA-Z'\s]"  
  , "").toLowerCase()
```

Punctuation stripped,  
everything lowercase so  
'Love' = 'love'



04

TRIE

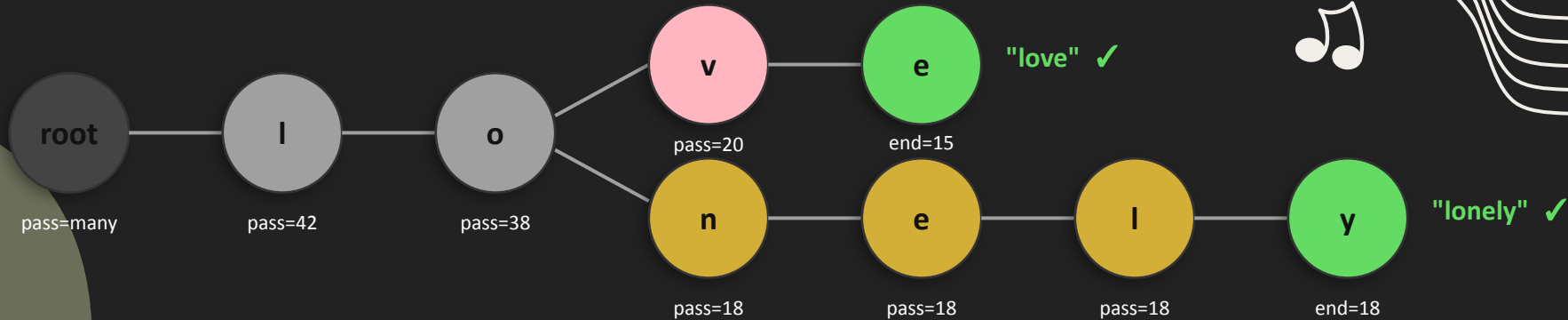
## Insert Line

```
trie.insertLine(  
  cleaned.split(  
    "\\s+"))
```

Line split into words, inserted  
into Trie + full line stored for  
guessing game



# Trie Structure



## Node Fields:

**Pass Count (pass)** - how many words travel through this node

**End Count (end)** - how many words end at this node (word frequency)

**Children** - Stored using a `HashMap<Character, Node>`

## Prediction Logic:

**mostLikelyNextChar()** - Follows prefix, pick child with highest passCount

**mostLikelyNextWord()** - follows highest passCount to a word end

**insertLine()** — stores full lyric lines and feeds each word into the Trie

# Big O Analysis

## Insertion

$O(L)$  efficiency where  $L$  is length of word being added:

```
for(char c : word.toCharArray()){  
    Node child = curr.children.get(c);  
    if(child == null){  
        child = new Node();  
        curr.children.put(c, child);  
    }  
}
```

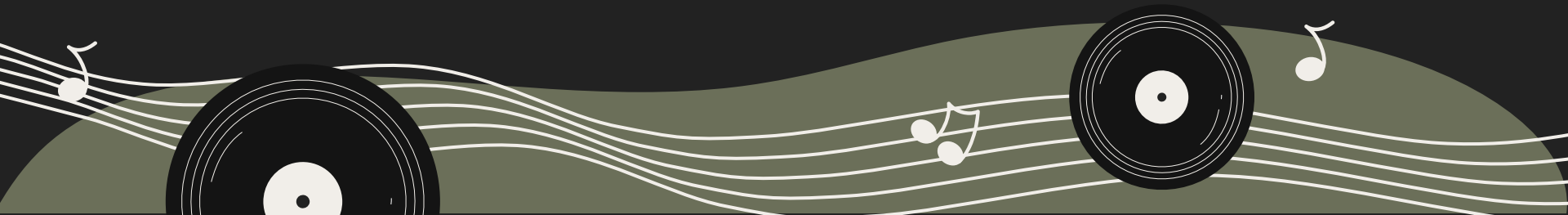
We can see from this code snippet that our method iterates over how long the word is while it is inputting said word into the Trie which is why this is  $O(L)$ .

## Contains

$O(L)$  efficiency where  $L$  is length of word being searched for:

```
for(char c : word.toCharArray()){  
    Node child = curr.children.get(c);  
    if(child == null)  
        return false;  
    curr = child;  
}
```

We can see from this code snippet that our method iterates over how long the word being searched is which is why this is  $O(L)$ .



# Big O Analysis cont.

## Next Char

O(L) efficiency where L is the length of the prefix given:

```
for(int i = 0; i < prefix.length(); i++){  
    char c = prefix.charAt(i);  
    curr = curr.children.get(c);  
    if(curr == null)  
        return '_';  
}
```

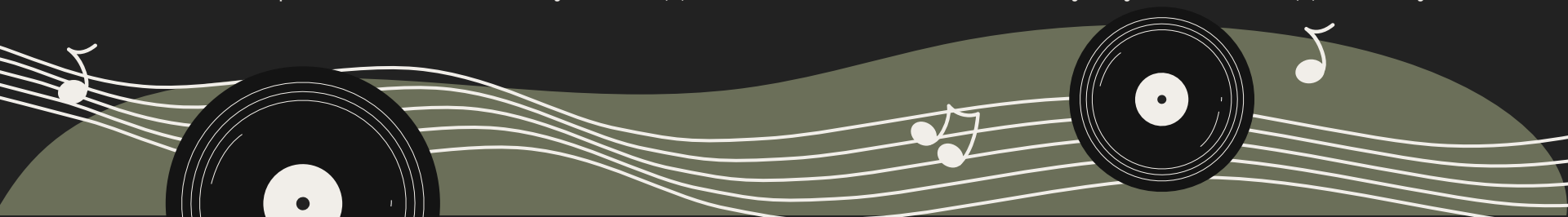
We can see from this code snippet that our method iterates over how long the prefix given is to find the node where the prefix resides which is why this is O(L)

## Next Word/s

O(L) efficiency where L is the length of the prefix given:

```
for(int i = 0; i < prefix.length(); i++){  
    char c = prefix.charAt(i);  
    curr = curr.children.get(c);  
    if(curr == null)  
        return "";  
}
```

Both next word and next 5 words uses the same loop structure to get to the current node where the prefix resides which is why they both have a O(L) efficiency



# Big O Analysis cont..

## Collect Words

$O(N)$  efficiency where  $N$  is the number of nodes in the Trie:

```
for (Map.Entry<Character, Node>entry
      :node.children.entrySet())
    collectWords(entry.getValue(),
        word+entry.getKey(),
        words,counts);
```

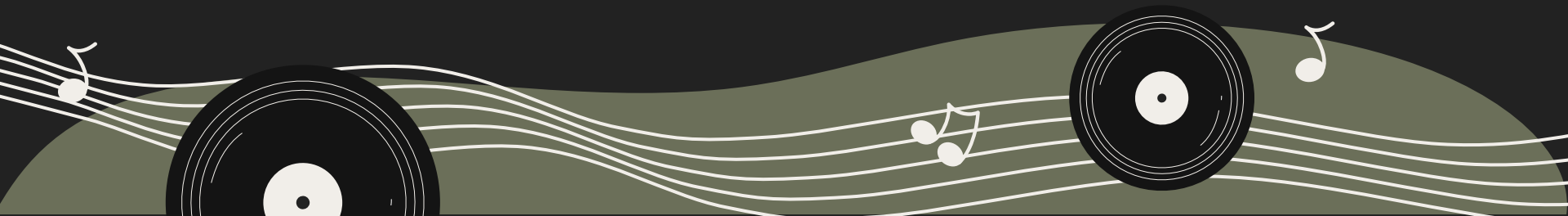
We can see from this code snippet that our method iterates over every node under a given prefix in a Trie which gives the  $O(N)$  efficiency.

## Guessing Game

$O(N)$  efficiency where  $N$  is the number of words in the song lyric chosen:

```
for(int x=0;x<arr.length;x++)
    arrNew[x] = arr[x];
```

To return a new array with the same words as a line in a song with one blank space for the guessing game, we must iterate through the song lyric to copy each word from an already existing array that has the lyrics to a new array causing this  $O(N)$  efficiency where  $N$  is the number of words in a line.



# Ethical Use Slide

- This product should not be used to plagiarize the lyrics of Kendrick Lamar and Taylor Swift and should only be used as a guiding tool
- This product is trained on a copyrighted materials of these artists meaning full songs should not be generated without permission of these artists
  - This tool is intended for educational purposes and inspiration
- Warning: the UI of this tool was partially created with the help of generative AI



# Video

Predictive Writing Assistant - Tayloria Kendrick

**Predictive Writing Assistant**

[View code](#) | [Watch demo to compute predictions](#)

Type here (green = in dataset, red = not found)

I

**Predictions:**

Next letter: -

Next word: -

Guessing Game - Fill in the blank!

Your guess:  [Submit](#) [New Line](#) [Ping New line to start](#)